EventForge

Smart Vendor Scheduling and Booking Platform for Multi-Vendor Events

DBMS Experiential Learning – Level 3 Project | Phase 1 (50% Review Package)

Prepared for: Phase 1 review

Project type: Two-member DBMS mini project

Database option: SQL + NoSQL

Status: Foundation and database design complete; application implementation planned for Phase 2

1. Executive summary

EventForge is a multi-vendor event planning platform that helps an organizer discover suitable vendors, check event-date availability, coordinate multiple services, and manage bookings without vendor schedule conflicts. Unlike a basic CRUD application, EventForge addresses a real scheduling problem: a vendor may offer several services, but the same vendor must not be assigned to overlapping events. The platform therefore combines relational integrity and transactional booking logic with flexible vendor discovery documents.

2. Problem statement

Event organizers commonly coordinate caterers, decorators, audio-visual teams, photographers, and other vendors through disconnected messages and spreadsheets. This creates duplicate bookings, unclear availability, inconsistent prices, missed payments, and difficulty comparing vendors. Vendors also lack a structured way to maintain service descriptions, portfolios, service areas, and FAQs.

3. Objectives

1. Allow organizers to create events and specify date, time, venue, guest count, and event type.
2. Let vendors publish services, capacities, prices, service areas, and flexible profile content.
3. Search and filter vendors by category, city, keywords, rating, capacity, and price.
4. Prevent overlapping bookings for the same vendor through database-level logic.
5. Track booking status, payments, reviews, and event-level summaries.
6. Demonstrate a justified SQL + NoSQL design with advanced database features.

4. Scope

In scope for the complete project

Organizer, vendor, and administrator roles; event creation; vendor/service discovery; availability slots; booking requests; conflict detection; payment status tracking; reviews; dashboards; keyword search; validation; secure configuration; and a responsive Next.js interface.

Included in Phase 1

Requirements analysis, architecture, database choice justification, ER diagram, relational schema, PostgreSQL DDL, indexes, view, trigger, stored function, sample DML, seed records, MongoDB document model, text index, aggregation example, and Phase 2 plan.

Deferred to Phase 2

Next.js pages and forms, API routes, authentication screens, role-based authorization middleware, live booking workflow, MongoDB synchronization, automated tests, deployment, and final screenshots.

5. Users and responsibilities

Role	Main responsibilities
Organizer	Create events, search vendors, request services, confirm bookings, view payments and reviews
Vendor	Maintain profile and services, publish availability, accept or reject booking requests
Admin	Approve vendors, monitor platform data, manage exceptions and reports

The two team members should both contribute to database and application work. A suggested division is: **Member 1 – PostgreSQL schema, transaction logic, API integration, booking screens; Member 2 – MongoDB discovery model, search/aggregation, UI dashboards, testing and documentation.** Both members review each other's work and commit individually to GitHub.

6. System architecture

The Next.js client will call protected Route Handlers. The API layer will validate input, use PostgreSQL for all booking and payment mutations, and query MongoDB for flexible vendor discovery content. Environment variables will hold connection strings and secrets. No credential will be hard-coded.

Organizer/Vendor browser

|

Next.js UI

|

Route Handlers / API layer

/ \

PostgreSQL MongoDB

(system of record) (discovery profiles)

7. Database selection and justification

PostgreSQL is mandatory and is appropriate for users, events, services, bookings, payments, and reviews because these records have relationships, constraints, and transaction requirements. MongoDB is appropriate for vendor portfolios, FAQs, tags, flexible service descriptions, and other descriptive content whose shape can vary between vendors. The boundary is explicit: MongoDB improves discovery, but a booking is valid only after PostgreSQL confirms the authoritative service and schedule state.

8. Relational schema summary

- `users(user_id PK, full_name, email UK, password_hash, role, created_at)`
- `events(event_id PK, organizer_id FK, event_name, event_type, event_date, start_time, end_time, venue_city, expected_guests, status)`
- `vendors(vendor_id PK, owner_user_id FK/UK, business_name UK, category, city, rating, approval_status)`
- `services(service_id PK, vendor_id FK, service_name, description, base_price, capacity, active)`
- `availability_slots(slot_id PK, service_id FK, slot_date, start_time, end_time, status)`
- `bookings(booking_id PK, event_id FK, service_id FK, agreed_price, booking_status, booked_at)`
- `payments(payment_id PK, event_id FK, booking_id FK, amount, payment_status, payment_method, paid_at)`
- `reviews(review_id PK, event_id FK, vendor_id FK, reviewer_id FK, rating, comment, created_at)`

The ER diagram is supplied separately in `diagrams/eventforge-er.png`.

9. Advanced database concepts demonstrated

1. **Transactions:** payment insertion and availability update are grouped in a transaction.

2. **View:** `event_booking_summary` provides event-level service count and committed amount.

3. **Stored function:** `search_services()` performs reusable full-text service search with optional city filtering.

4. **Trigger:** `prevent_overlapping_vendor_booking()` rejects overlapping bookings for the same vendor.

5. **Indexing:** date/city, vendor/service, availability status, booking status, and GIN full-text indexes are defined.

6. **Full-text search:** PostgreSQL `to_tsvector`/`plainto_tsquery` searches service descriptions.

7. **NoSQL aggregation:** MongoDB text-score and city-filter pipeline returns discovery cards.

10. Integrity, validation, and security

Database-level checks enforce valid roles, statuses, positive prices and capacities, valid ratings, non-empty time ranges, unique emails, unique vendor businesses, and referential integrity. API validation will repeat user-facing checks before database calls. Passwords will be stored only as Argon2 or bcrypt hashes. Connection strings and session secrets will be environment variables. Parameterized queries or a trusted ORM will be used to prevent injection.

11. Representative operations

The SQL scripts demonstrate `CREATE`, `DROP`, keys, constraints, indexes, view creation, `INSERT`, `UPDATE`, `DELETE` guidance, `SELECT`, joins, grouping, `HAVING`, ordering, aggregates, subqueries, a transaction, full-text search, a stored function, and a trigger. The MongoDB script demonstrates collection creation, unique/compound/text indexes, flexible documents, and aggregation.

12. Phase 2 implementation plan

1. Initialize Next.js application and environment configuration.
2. Add Prisma or parameterized PostgreSQL data-access layer and MongoDB client.
3. Implement login, password hashing, sessions, and role checks.
4. Build organizer event forms and vendor profile/service forms.
5. Build search, filters, service detail, availability, and booking confirmation flows.
6. Expose conflict-safe booking transaction through an API route.
7. Add dashboards for organizer, vendor, and admin roles.
8. Add validation, error states, audit logging, tests, screenshots, and deployment instructions.

13. Phase 1 review checklist

- ☑ Unique real-world project idea beyond basic CRUD
- ☑ SQL + NoSQL choice justified
- ☑ ER diagram and relational schema
- ☑ Primary keys, foreign keys, constraints, and indexes
- ☑ DDL and seed DML
- ☑ At least four advanced features
- ☑ MongoDB data model, indexes, and aggregation strategy
- ☑ Architecture and security plan
- ☑ Clear Phase 2 boundary
- ☐ Complete Next.js interface and API integration

- ☐ Final testing, deployment, screenshots, and demo

14. Conclusion

The Phase 1 package establishes a credible database-first foundation for EventForge. The design goes beyond CRUD by modelling a multi-vendor scheduling constraint and enforcing it at the database layer. The next milestone will connect this foundation to a functional Next.js application and demonstrate the end-to-end organizer and vendor workflows.